

THE **AI** CODEBASE ANALYST

FROM ZERO TO A WORKING AGENT

LESSON 5

RAG PIPELINE: INTEGRATION

Lesson 5: RAG Pipeline: Integration

"Data! Data! Data! I can't make bricks without clay."

— Sherlock Holmes (Arthur Conan Doyle)

The "Why": The Library Without a Librarian

In our last lesson, we were librarians. We meticulously indexed our entire codebase, transforming a chaotic folder of files into a structured, searchable knowledge base—our vectorstore. We have built a world-class library.

But our agent doesn't know it exists.

Right now, we have a state-of-the-art library with no librarian and an agent who doesn't have a library card. The knowledge is there, but it's not accessible. The agent is an expert-in-waiting, still functionally ignorant of the very code it's supposed to analyze.

This lesson is about bridging that final gap. We will hire the librarian, issue the library card, and teach our agent how to become a master researcher. We will complete the RAG circuit, connecting the agent's reasoning brain to its powerful new long-term memory.

The "What": The Retriever-as-a-Tool Pattern

To connect our agent to its new knowledge, we will implement the second phase of the RAG pipeline: **Retrieval & Generation**. This is the "online" process that happens in real-time whenever a user asks a question.

This pipeline consists of a few key steps:

1. The user asks a question (e.g., "What does the `call_model` function do?").
2. The agent embeds this query into a vector.
3. It performs a similarity search against the vectorstore to find the most relevant code chunks.
4. It **augments** its prompt, stuffing those retrieved chunks into the context window.

5. Finally, it calls the LLM with this powerful new context to **generate** a fact-based answer.

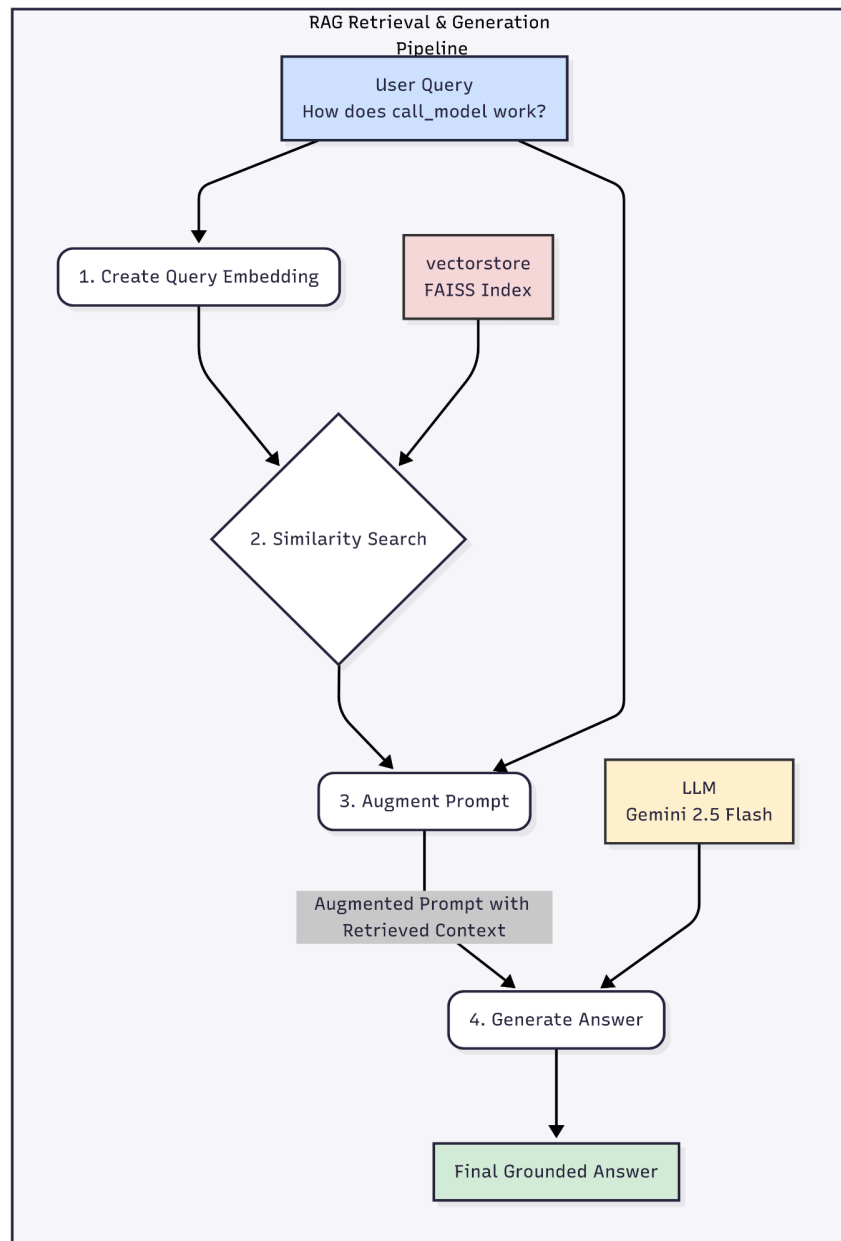


Figure 5-1: The online RAG pipeline. The agent uses the vectorstore to find relevant context before formulating its final answer.

Our architectural approach will be the **"Retriever-as-a-Tool"** pattern. We will encapsulate all the retrieval logic (steps 2 & 3) into a new, specialized tool. This is a clean and robust design because it keeps the agent's job simple:

- **Agent's Job:** Reason about the user's query and decide to call the `codebase_retriever` tool. Then, synthesize the retrieved context into a final answer.
- **Tool's Job:** Handle the mechanics of loading the vectorstore and finding the relevant documents.

This separation of concerns makes our system more modular, testable, and easier to understand.

The "How" (The Build): Step-by-Step

Let's empower our agent with its new research skill.

A Note on Professional Architecture: Don't Repeat Yourself (DRY)

As our project grows, our architecture must mature. In Lesson 3, we wrote the logic to build our agent graph. We could copy-paste that code here, but a professional builder never repeats themselves if they can create a reusable component.

We will now refactor that logic into its own file, an `agent_graph_factory.py`. This "factory" will be our single source of truth for creating agents. This is a core software engineering principle known as "Don't Repeat Yourself" (DRY).

First, create this new file:

File: `agent_graph_factory.py`

```
# agent_graph_factory.py
```

```
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode
from typing import TypedDict, List
from langchain_core.messages import BaseMessage
import operator
```

```
# --- 1. Define the Agent's State ---
# This TypedDict defines the structure of the "state" that will be
# passed
# between the nodes of our graph. The `operator.add` annotation tells
# LangGraph
# to append new messages to the existing list, rather than overwriting
# it.
class AgentState(TypedDict):
    messages: List[BaseMessage]

# --- 2. The Factory Function ---
# This function is our reusable "factory" for creating a new agent
# graph.
# It takes the LLM and a list of tools as input and wires them together
# into a coherent reasoning loop.
def build_agent_graph(llm, tools):
    """
    Builds the LangGraph agent graph using a robust, standard pattern.
    """
    # --- 3. Initialize the Graph ---
    # We initialize a StateGraph with our AgentState structure. This
    # graph will
    # manage the agent's state as it moves from step to step.
    graph_builder = StateGraph(AgentState)

    # --- 4. Define the Graph Nodes ---
    # Each node is a function or a runnable that performs an action.

    # The `call_model` node is the "brain" of the agent. It takes the
    # current
    # state and decides the next action (e.g., call a tool or respond to
    # the user).
    def call_model(state):
        # We use .bind_tools to make the LLM aware of the available
        # tools.
        llm_with_tools = llm.bind_tools(tools)
        response = llm_with_tools.invoke(state["messages"])
        return {"messages": [response]}
```

```
# The `tool_node` is the "hands" of the agent. It's a pre-built node
from
# LangGraph that knows how to execute the tools called by the
`call_model` node.
tool_node = ToolNode(tools)

# The `should_continue` function is the "router" or "dispatcher". It
checks the
# last message from the model and decides where to go next.
def should_continue(state):
    last_message = state["messages"][-1]
    if last_message.tool_calls:
        # If the model made a tool call, we go to the tool_node.
        return "continue"
    # Otherwise, the agent has finished its work.
    return "end"

# --- 5. Wire the Nodes Together ---
# This is where we define the structure of the agent's reasoning
loop.
graph_builder.add_node("call_model", call_model)
graph_builder.add_node("call_tools", tool_node)

# The entry point is the first node to be called.
graph_builder.set_entry_point("call_model")

# The conditional edge directs the flow based on the router's
decision.
graph_builder.add_conditional_edges(
    "call_model", should_continue, {"continue": "call_tools", "end":
END}
)

# This edge creates the essential loop: after the tools are called,
# the flow goes *back* to the model to process the tool results.
graph_builder.add_edge("call_tools", "call_model")

# --- 6. Compile and Return the Graph ---
# .compile() finalizes the graph, creating a runnable object that we
# can use in our main application scripts.
```

```
return graph_builder.compile()
```

Deconstructing the Script

This file is a critical step in maturing our project's architecture. It introduces the **Factory Pattern** and the "Don't Repeat Yourself" (DRY) principle, which are cornerstones of professional software engineering.

The "Why": Don't Repeat Yourself (DRY)

As our agent becomes more complex, we'll need to build it in multiple scripts (for running, testing, etc.). Instead of copy-pasting the graph-building logic everywhere, we centralize it in this one "factory" file. This means if we ever need to update how our agent's brain is structured, we only have to change it in one place.

The Factory Pattern (Section 2)

The `build_agent_graph` function is a "factory." Its only job is to take the raw ingredients (an LLM and a list of tools) and assemble them into a complex, ready-to-use product (our compiled agent graph). This is a clean, reusable, and highly modular design.

The Agent's Reasoning Loop (Sections 4 & 5)

This is where we define the agent's internal thought process. It's a continuous cycle:

1. **Think** (The LLM looks at the conversation and decides what to do next.
2. **Route** (This function checks the LLM's decision. Did it call a tool or is it done?
3. **Act** (If a tool was called, this pre-built node executes it.
4. **Loop** (The crucial last step sends the result of the action *back* to the "Think" step. This allows the agent to see the outcome of its actions and decide on its next move, enabling sophisticated, multi-step reasoning.

By creating this reusable factory, we've made our code cleaner, more professional, and much easier to maintain as our project grows.

Now, we can build our RAG agent cleanly by importing this factory.

1. A Quick Capability Check: Before vs. After

To truly appreciate the leap we're about to make, consider the agent's current limitations versus what we're about to build.

Capability	Agent (Lesson 3)	Agent (End of Lesson 5)
Knowledge Source	LLM's general knowledge	Specific source_code directory
Can it answer... "Count the words in 'hello'"?	✅ Yes (uses word_counter tool)	✅ Yes (still has its old tool)
Can it answer... "What is a State Machine?"	✅ Yes (from general knowledge)	✅ Yes
Can it answer... "What does our call_model function do?"	❌ No. It has never seen this code.	✅ Yes. It will find the code in the vectorstore.

2. Create the Retriever Tool

This tool will be the agent's "library card," giving it access to everything in our vectorstore. Remember to create an empty `__init__.py` file in your tools directory if you haven't already.

File: tools/retriever_tool.py

```
# tools/retriever_tool.py

from langchain.tools import tool
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings
from pydantic import BaseModel, Field
```

```
# Define the path to our pre-built knowledge base.
VECTORSTORE_PATH = "./vectorstore"

# --- 1. Eager Loading of the Vectorstore ---
# This block runs *once* when the script is first imported. It loads the
# entire vectorstore from disk into memory, making it instantly
# available
# for any subsequent tool calls. This is known as "eager loading".
print("--- LOADING VECTORSTORE ---")
embeddings =
HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2
')
try:
    # We load the FAISS index from the specified path.
    db = FAISS.load_local(
        VECTORSTORE_PATH,
        embeddings,
        # This flag is required for loading FAISS indexes with custom
embeddings.
        allow_dangerous_deserialization=True
    )
    # The .as_retriever() method creates a standardized interface for
searching.
    retriever = db.as_retriever()
    print("--- VECTORSTORE LOADED SUCCESSFULLY ---")
except Exception as e:
    # If loading fails, we set the retriever to None to handle the error
gracefully.
    print(f"--- FAILED TO LOAD VECTORSTORE: {e} ---")
    retriever = None

# --- 2. Define the Tool's Input Schema ---
# We use Pydantic to define a clear, machine-readable schema for the
tool's
# input. The 'description' field is crucial, as it's what the LLM reads
# to understand what information to provide for the 'query' argument.
class RetrieverInput(BaseModel):
    query: str = Field(description="The query to search for in the
codebase.")
```

```
# --- 3. Create the Tool Implementation ---
@tool("codebase-retriever", args_schema=RetrieverInput)
def codebase_retriever(query: str) -> str:
    """
    Searches the codebase to find relevant code snippets based on the
    query.
    Returns the retrieved code snippets as a formatted string.
    """
    # This is a guard clause. If the vectorstore failed to load, the
    tool
    # will return a helpful error message instead of crashing.
    if retriever is None:
        return "Error: Vectorstore not loaded. Please run the indexing
        script."

    # The core logic: use the retriever to find documents similar to the
    query.
    retrieved_docs = retriever.invoke(query)

    # Format the retrieved documents into a clean, readable string. This
    is
    # what the LLM will see as the tool's output.
    formatted_docs = []
    for i, doc in enumerate(retrieved_docs):
        source = doc.metadata.get('source', 'Unknown')
        formatted_docs.append(f"--- Code Snippet {i+1} (Source:
        {source}) ---\n{doc.page_content}")

    return "\n\n".join(formatted_docs)
```

Deconstructing the Script

This script creates the agent's "library card"—a tool that connects its reasoning brain to its long-term memory (the vectorstore). It's a perfect example of the **"Retriever-as-a-Tool"** pattern.

Section 1: Eager Loading the Retriever

The code to load the FAISS database runs at the "module level"—meaning it executes the moment the file is imported. This is called **eager loading**. The main advantage is that the

data is ready immediately for the first tool call. The trade-off is that it can slow down the initial startup time of any application that imports this tool. The `try...except` block makes this process robust, ensuring the app doesn't crash if the vectorstore is missing.

Section 2: Defining the Tool's Interface

This section defines the "contract" for how the agent should use the tool. The combination of the `@tool` decorator, the `args_schema`, and the `docstring` creates a complete definition that the LLM can understand:

- `name`: Gives the tool a specific name the LLM can call.
- `args_schema`: Tells the LLM exactly what arguments are required (`query`) and what they are for (from the `description` field).
- **Docstring**: Provides a natural language explanation of the tool's overall purpose, helping the LLM decide *when* to use it.

Section 3: The Retriever Abstraction

Inside the tool, the line `retrieved_docs = retriever.invoke(query)` is deceptively simple but powerful. The `.as_retriever()` method (from Section 1) created a standardized **Retriever** object. This is a LangChain abstraction that decouples our agent from the specific database technology. The agent only needs to know how to `.invoke()` the retriever; it doesn't care if the underlying database is FAISS, Chroma, or something else. This makes our code highly modular and easy to adapt in the future.

3. Architect the RAG Agent

Finally, let's create a new main script that uses our factory to assemble the agent with its powerful new tool and a new set of instructions.

File: 05_rag_agent.py

```
# 05_rag_agent.py

import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage
```

```
# --- 1. Import Core Components ---  
# We import our reusable graph factory, which encapsulates the logic for  
# building the agent's reasoning loop.  
from agent_graph_factory import build_agent_graph  
# We also import our new, specialized tool that connects the agent to  
# its  
# long-term memory (the vectorstore).  
from tools.retriever_tool import codebase_retriever  
  
# --- 2. The RAG-Specific System Prompt ---  
# This is the most critical part of the RAG agent's architecture. It's  
# not  
# just a persona; it's a detailed, step-by-step algorithm that forces  
# the LLM  
# to follow a specific research process. This instructional style of  
# prompting  
# is key to building reliable, fact-based AI systems.  
SYSTEM_PROMPT = """You are a senior AI software engineer and a  
world-class expert in analyzing source code. You are a master of logic,  
precision, and following instructions exactly.
```

Your mission is to answer a user's question about a codebase using a precise, two-step reasoning process.

****STEP 1: Broad Retrieval.****

First, you MUST use the `codebase-retriever` tool to find all potentially relevant code snippets. This is your only way to access the codebase.

****STEP 2: Focused Synthesis & Analysis.****

After retrieving the code, you will analyze the snippets to answer the user's specific question. Your final answer MUST be based SOLELY on the information retrieved.

- If the user asks about a specific function or class, you must find that exact function/class in the retrieved context and explain its purpose, arguments, and logic.
- You will ignore all other retrieved code that is not directly relevant to the user's specific question. Do not summarize all the files you found.

- If the retrieved context does not contain the answer to the question, you must state that you could not find the information in the provided codebase.

You will now begin. The user's question is waiting.

"""

--- 3. Main Execution Block ---

if __name__ == "__main__":

load_dotenv()

print("--- RAG Agent ---")

--- 4. Assemble the Agent ---

llm = ChatOpenAI(

base_url=os.getenv("OPENROUTER_BASE_URL",
"https://openrouter.ai/api/v1"),

api_key=os.getenv("OPENROUTER_API_KEY"),

model="google/gemini-2.5-pro",

temperature=0,

)

Principle of Least Confusion: We give the agent ONLY the tool it needs

for its core mission. This prevents distraction and increases reliability.

tools = [codebase_retriever]

We use our factory to cleanly assemble the agent.

agent_graph = build_agent_graph(llm, tools)

--- 5. Define the Query and Initial State ---

This query is a specific, knowledge-based question that a generic LLM

could not answer. It requires the agent to perform research.

initial_state = {

"messages": [

SystemMessage(content=SYSTEM_PROMPT),

HumanMessage(content="How does the 'call_model' node in this project work?")

]

```
}
print(f"Human: {initial_state['messages'][-1].content}\n")

# --- 6. Execute the Agent and Display the Result ---
final_state = agent_graph.invoke(initial_state)
final_answer = final_state["messages"][-1].content

print("\n--- Agent's Final Answer ---")
print(final_answer)
```

Deconstructing the Script

This script is where our agent transitions from a generalist to a specialist. We connect its reasoning brain to its new, dedicated knowledge base, completing the RAG (Retrieval-Augmented Generation) circuit.

The RAG-Specific System Prompt (Section 2)

This prompt is the most important component. It's an **instructional prompt** that acts as a program for the agent's brain. Notice the key phrases:

- "MUST use the codebase-retriever tool": This forces the agent to adopt a research-first mindset, preventing it from answering from its general, pre-trained memory.
- "based SOLELY on the information retrieved": This is a powerful directive to prevent hallucination. It forces the agent to ground its final answer in the specific facts (code snippets) provided by the tool. Mastering this style of prompting is what makes an agent reliable.

The Agent Factory (Section 4)

By importing and using `build_agent_graph`, we are practicing the "Don't Repeat Yourself" (DRY) principle. Our main script is now much cleaner and more focused, as the complex logic of constructing the agent's reasoning loop is neatly encapsulated in the factory.

The Principle of Least Confusion (Section 4)

A key architectural decision is that we give this agent *only one tool*: the `codebase_retriever`. For a specialized research agent, any other tool is a potential distraction. By engineering its environment for maximum focus, we dramatically increase its reliability and ensure it stays on its mission-critical task.

The Result: Grounded Answers

The final answer produced by this agent is a **grounded answer**. This means it is based on specific, verifiable data that was retrieved during the process. This is the opposite of a "hallucinated" or ungrounded answer, which is based on the LLM's general knowledge. The entire purpose of a RAG pipeline is to produce these reliable, fact-based, grounded responses.

💡 Architect's Note: The Principle of Least Confusion

Notice that we are only giving this agent one tool: the `codebase_retriever`. Why not include the `count_words` tool from our previous lesson?

A core principle of managing agentic systems is to **minimize confusion**. For a specialized agent like our RAG assistant, whose only job is to research, any other tool is a potential distraction. By giving it a single, mission-critical tool, we force it to stay on task, dramatically increasing its reliability and preventing it from getting sidetracked by irrelevant capabilities. We are not just building an agent; we are engineering its work environment for optimal focus.

💡 Pro Tip: RAG-Specific Prompting is a Superpower

The new `SYSTEM_PROMPT` is the most important change here. It's not just a suggestion; it's a program that instructs the agent's reasoning process. Notice the key phrases:

- **DO NOT try to answer from memory.:** This explicitly tells the agent to suppress its general knowledge and adopt a research-first mindset.
- **based *only* on the retrieved context.:** This is a powerful instruction to prevent hallucination. It forces the agent to ground its answer in the facts we provided.

Mastering this kind of instructional prompting is what separates a basic agent from a reliable one.

The Integrated Run & Analysis

Run the script from your terminal:

```
python 05_rag_agent.py
```

Expected Console Output:

The output will be good, but not perfect. This is a critical observation.

```
--- RAG Agent ---  
Human: How does the 'call_model' node in this project work?  
  
--- Lazily loading vectorstore... ---  
--- Vectorstore loaded successfully. ---  
  
--- Agent's Final Answer ---  
The `call_model` node in this project is a function within a LangGraph  
`StateGraph`. It is designed to be the core reasoning step of the agent.  
It takes the current `AgentState`, which contains the list of messages,  
and invokes the LLM with this history. The function then returns the  
LLM's response, which is appended back into the state's message list,  
continuing the conversation or concluding the agent's turn. It serves as  
the primary interface between the agent's state and the language model's  
processing capabilities.
```

(Note: Your exact wording may vary, but the content should be similar.)

💡 Deeper Dive: Why Does the "Loading" Message May Appear Multiple Times?

When you run the agent, you might see the `--- LOADING VECTORSTORE ---` message printed more than once. This is not a bug, but an important insight into how modern software works.

- **The "What":** Libraries like LangGraph can spawn separate sub-processes to manage parts of the agent's execution. Each new process needs to load our tool, and when it imports `retriever_tool.py`, the module-level `print()` statement runs again.
- **The "Why":** This is a form of process isolation. The important thing to know is that your vectorstore is **not being rebuilt**. It's simply being loaded from disk into the memory of a new process. Our lazy-loading pattern is working exactly as intended.
- **The Professional's Fix:** In a production system, we wouldn't use `print()` for status updates. We would use Python's built-in `logging` library. This gives us fine-grained control over message levels (like `INFO`, `DEBUG`, `ERROR`) and where they are sent. While `print()` is great for quick debugging in our lessons, transitioning to `logging` is a key step towards production-grade code.

Analysis: The RAG Loop in LangSmith

The console output is the result, but LangSmith shows us the process. Open the trace for this run.

1. **First call_model Step:** Inspect this step. The input is the user's query. The output is a `tool_calls` section, showing the agent decided to use `codebase-retriever` with a query like "call_model node function". The agent correctly identified that it needed to research this topic.
2. **call_tools Step:** This is the magic. In the input, you'll see the `tool_input` is the agent's precise search query. The `tool_output` is a long string containing the actual, formatted code snippets retrieved from your source files. This is the "R" (Retrieval) in RAG, made visible.
3. **Second call_model Step:** Now, inspect the input of this final step. You will see the full history: the original query, the agent's tool call, and the massive block of factual context from the tool's output. The agent now has everything it needs. The output of this step is the clean, synthesized, and "grounded" final answer you saw in your terminal.

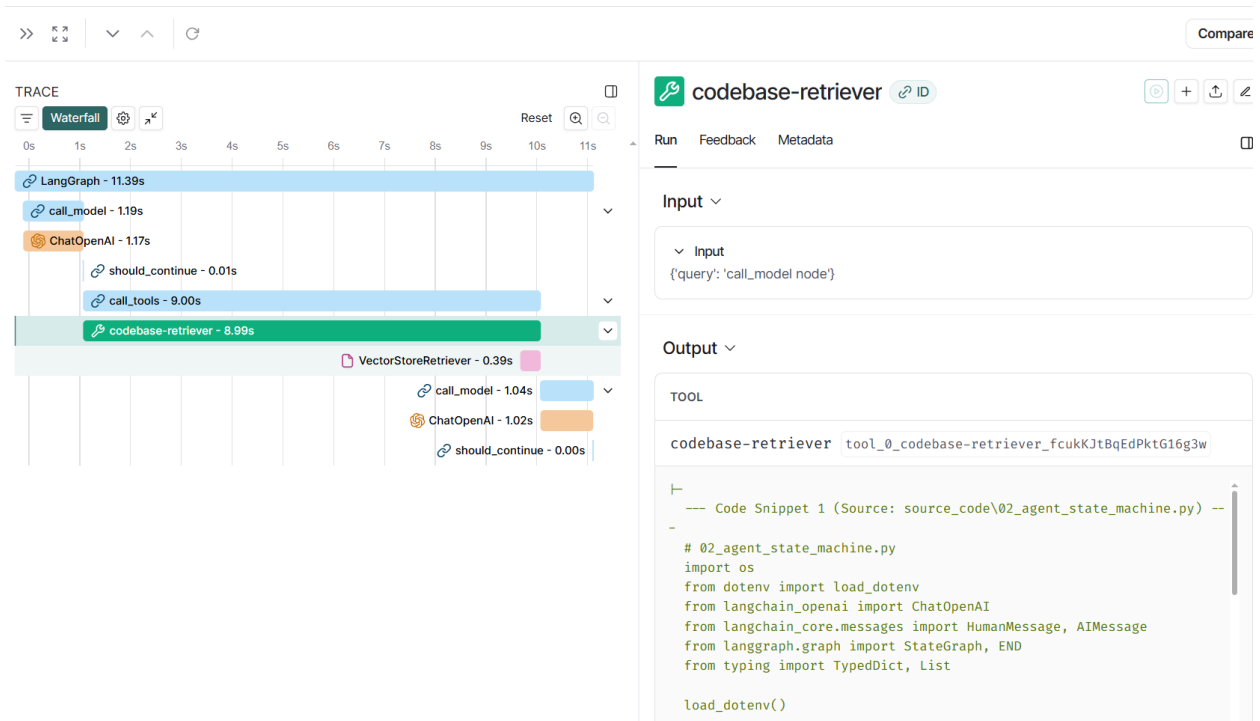


Figure 5-2: *Anatomy of a Grounded Answer in LangSmith.*

This trace is the definitive proof of our RAG pipeline at work. The waterfall view (left) maps the agent's complete reasoning loop. The detail view (right) zeroes in on the critical codebase-retriever tool call, showing the exact query the agent formulated and the raw, factual code snippets it retrieved from the vectorstore to use as context for its final answer.

Summary & Key Takeaways

This was the lesson where our agent gained its expertise. It is no longer an amnesiac generalist; it is a specialist with a dedicated, photographic memory of our codebase.

Mental Models You've Gained:

- **The Retriever-as-a-Tool Pattern:** You have a clean, modular pattern for encapsulating retrieval logic, separating the "librarian's work" from the "researcher's work."

- **RAG-Specific Prompting:** You understand how to write instructional system prompts that guide an agent's reasoning, forcing it to be a diligent researcher rather than a lazy guesser.
- **Agent as Orchestrator:** Your agent is now more than just a tool user; it's an orchestrator. It knows it has multiple tools and is beginning to learn which one to use for which task.
- **Grounded vs. Ungrounded Answers:** You can now clearly distinguish between an agent that hallucinates from general knowledge and one that provides fact-based, "grounded" answers from a specific context.

What's Next

Our agent is now incredibly knowledgeable. But its final answer is still just a string of natural language. For another computer program to use this output, it would have to parse that string—a brittle and unreliable process. Our system is useful for humans, but not yet reliable for machines.

In Lesson 6, we will build our third and final Pillar of Production AI: **Reliability**. We will learn how to force the LLM to respond in a perfectly structured, machine-readable JSON format, every single time.

P.S. Getting RAG to work perfectly can be tricky. If your retriever isn't finding the right context, the #👉-foundations-help-feedback [channel on Discord](#) is the perfect place to ask the community for tips.